CLASS ACTIVITY – 2

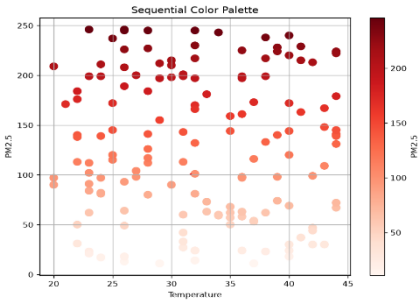
Scenario 1: Smart City Air Quality Monitoring

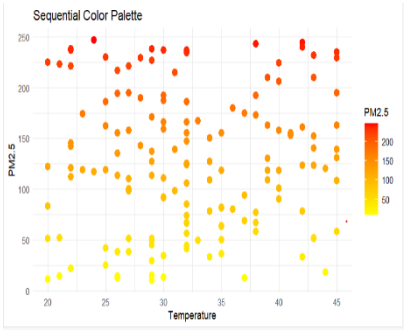
Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Generate sample dataset 	
np.random.seed(42)
n = 150
data = {
"Location": [f"Area_{i}" for i in range(1, n + 1)],
"PM2.5": np.random.randint(10, 250, n),
"NO2": np.random.randint(5, 150, n),
"CO": np.round(np.random.uniform(0.1, 5.0, n), 2),
"Temperature": np.random.randint(20, 45, n),
"Humidity": np.random.randint(30, 95, n)
}
df = pd.DataFrame(data)
# Save dataset 	
df.to_csv("air_quality.csv", index=False)
# ---------- Sequential Palette ---------- 	
plt.figure(figsize=(8,6))
scatter = plt.scatter(df["Temperature"], df["PM2.5"],
c=df["PM2.5"],
cmap="Reds",
s=80)
plt.colorbar(scatter, label="PM2.5")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.title("Sequential Color Palette")
plt.grid(True)
plt.show()
plt.figure(figsize=(8,6))
scatter = plt.scatter(df["Temperature"], df["PM2.5"],
c=df["PM2.5"],
cmap="coolwarm",
s=80)
plt.colorbar(scatter, label="PM2.5")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.title("Diverging Color Palette")
plt.grid(True)
plt.show()
df["Pollution Level"] = pd.cut(
df["PM2.5"],
	library(ggplot2)
	
	set.seed(42)
	
	n <- 150
	
	air <- data.frame(
	Location = paste("Area",1:n,sep="_"),
	PM2.5 = sample(10:250,n,replace=TRUE),
	NO2 = sample(5:150,n,replace=TRUE),
	CO = round(runif(n,0.1,5.0),2),
	Temperature = sample(20:45,n,replace=TRUE),
	Humidity = sample(30:95,n,replace=TRUE)
	)
	
	write.csv(air,"air_quality.csv",row.names=FALSE)
	
	# Sequential Palette
	ggplot(air,
	aes(x=Temperature,
	y=PM2.5,
	color=PM2.5))+
	geom_point(size=3)+
	
	scale_color_gradient(low="yellow",high="red")+
	ggtitle("Sequential Color Palette")+
	theme_minimal()
	
	# Diverging Palette
	ggplot(air,
	aes(x=Temperature,
	y=PM2.5,
	color=PM2.5))+
	geom_point(size=3)+
	scale_color_gradient2(low="blue",
	mid="white",
	high="red",
	midpoint=120)+
	ggtitle("Diverging Color Palette")+
	theme_minimal()
	
	# Qualitative Palette
	air$PollutionLevel <- cut(
	air$PM2.5,
	breaks=c(0,50,100,200,300),
	labels=c("Low","Moderate","High","Very High")

bins=[0,50,100,200,300], **labels=["Low","Moderate","High","Very High"]** **)** **colors = {** **"Low":"green",** **"Moderate":"yellow",** **"High":"orange",** **"Very High":"red"** **}** **plt.figure(figsize=(8,6))** **for level in colors:** **subset = df[df["Pollution Level"] == level]** **plt.scatter(subset["Temperature"],** **subset["PM2.5"],** **color=colors[level],** **s=80,** **label=level)** **plt.xlabel("Temperature")** **plt.ylabel("PM2.5")** **plt.title("Qualitative Color Palette")** **plt.legend()** **plt.grid(True)** **plt.show()** **print(df.head())**	**)** **ggplot(air,** **aes(x=Temperature,** **y=PM2.5,** **color=PollutionLevel))+** **geom_point(size=3)+** **ggtitle("Qualitative Color Palette")+** **theme_minimal()** **head(air)**

Output **Python output** **R output**





Scenario 2: Earthquake Risk Analysis

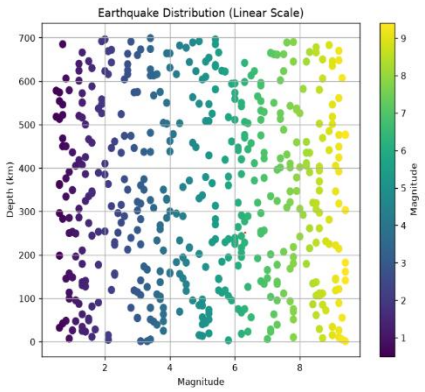
Python	R

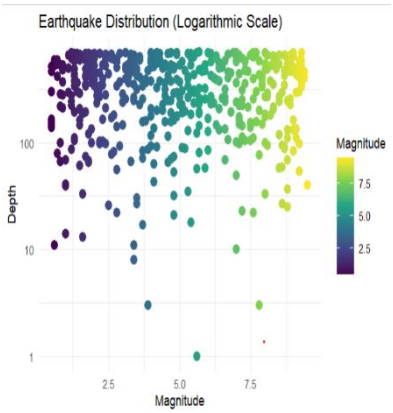
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Generate sample earthquake dataset 	
np.random.seed(42)
n = 500
data = {
"Magnitude": np.round(np.random.uniform(0.5, 9.5, n), 1),
"Depth": np.random.randint(1, 701, n),
"Latitude": np.round(np.random.uniform(-90, 90, n), 2),
	library(ggplot2)
	
	set.seed(42)
	
	n <- 500
	
	earthquake <- data.frame(
	Magnitude = round(runif(n,0.5,9.5),1),
	Depth = sample(1:700,n,replace=TRUE),
	Latitude = round(runif(n,-90,90),2),
	Longitude = round(runif(n,-180,180),2),
	Aftershocks = sample(0:150,n,replace=TRUE)

```
   "Longitude": np.round(np.random.uniform(-180, 180, n), 2),
   "Aftershocks": np.random.randint(0, 150, n)
}
df = pd.DataFrame(data)
# Save dataset
df.to_csv("earthquake_data.csv", index=False)
# -------- Linear Scale --------
plt.figure(figsize=(8,6))
plt.scatter(df["Magnitude"], df["Depth"],
            c=df["Magnitude"],
            cmap="viridis",
            s=50)
plt.colorbar(label="Magnitude")
plt.xlabel("Magnitude")
plt.ylabel("Depth (km)")
plt.title("Earthquake Distribution (Linear Scale)")
plt.grid(True)
plt.show()
# -------- Logarithmic Scale --------
plt.figure(figsize=(8,6))
plt.scatter(df["Magnitude"], df["Depth"],
            c=df["Magnitude"],
            cmap="plasma",
            s=50)
plt.yscale("log")
plt.colorbar(label="Magnitude")
plt.xlabel("Magnitude")
plt.ylabel("Depth (Log Scale)")
plt.title("Earthquake Distribution (Logarithmic Scale)")
plt.grid(True)
plt.show()
print(df.head())
```

```
)

write.csv(earthquake,
          "earthquake_data.csv",
          row.names=FALSE)

# Linear Scale Plot
ggplot(earthquake,
       aes(x=Magnitude,
           y=Depth,
           color=Magnitude))+
  geom_point(size=3)+
  scale_color_viridis_c()+
  ggtitle("Earthquake Distribution (Linear Scale)")+
  theme_minimal()

# Logarithmic Scale Plot
ggplot(earthquake,
       aes(x=Magnitude,
           y=Depth,
           color=Magnitude))+
  geom_point(size=3)+
  scale_y_log10()+
  scale_color_viridis_c()+
  ggtitle("Earthquake Distribution (Logarithmic Scale)")+
  theme_minimal()

head(earthquake)
```

Output





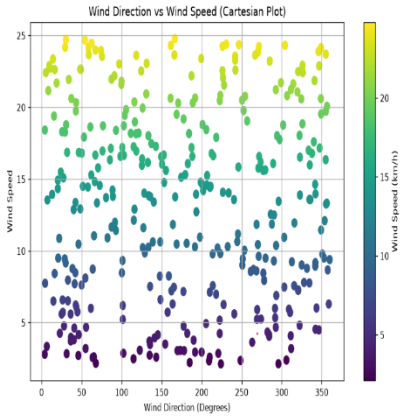
Scenario 3: Wind Direction Analysis for Airport Operations

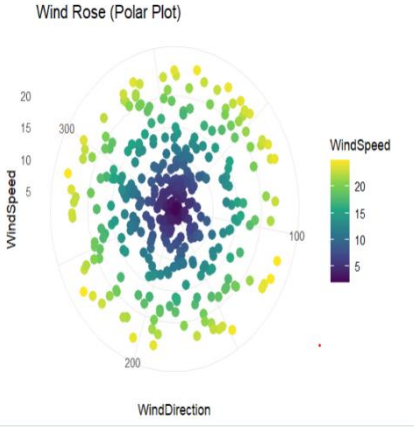
Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate sample dataset 	
np.random.seed(42)
 	
n = 365
 	
wind_speed = np.random.uniform(2, 25, n)
wind_direction = np.random.uniform(0, 360, n)
 	
months = np.random.choice([
"Jan","Feb","Mar","Apr","May","Jun",
"Jul","Aug","Sep","Oct","Nov","Dec"
], n)
 	
df = pd.DataFrame({
"Month": months,
"WindSpeed": wind_speed,
"WindDirection": wind_direction
})
 	
df.to_csv("wind_data.csv", index=False)
 	
# ---------- Cartesian Plot ---------- 	
plt.figure(figsize=(10,6))
 	
plt.scatter(df["WindDirection"],
df["WindSpeed"],
c=df["WindSpeed"],
cmap="viridis",
s=60)
 	
plt.colorbar(label="Wind Speed (km/h)")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed")
plt.title("Wind Direction vs Wind Speed (Cartesian Plot)")
plt.grid(True)
 	
plt.show()
 	
# ---------- Polar Plot (Wind Rose Style) ---------- 	
theta = np.deg2rad(df["WindDirection"])
 	
fig = plt.figure(figsize=(8,8))
	library(ggplot2)
	
	set.seed(42)
	
	n <- 365
	
	wind <- data.frame(
	Month =
	sample(c("Jan","Feb","Mar","Apr","May","Jun",
	"Jul","Aug","Sep","Oct","Nov","Dec"),
	n,
	replace=TRUE),
	WindSpeed = runif(n,2,25),
	WindDirection = runif(n,0,360)
	)
	
	write.csv(wind,"wind_data.csv",row.names=FALSE)
	
	# Cartesian Plot
	
	ggplot(wind,
	aes(x=WindDirection,
	y=WindSpeed,
	color=WindSpeed))+
	
	geom_point(size=3)+
	
	scale_color_viridis_c()+
	
	ggtitle("Wind Direction vs Wind Speed")+
	
	theme_minimal()
	
	# Polar Coordinate Plot
	
	ggplot(wind,
	aes(x=WindDirection,
	y=WindSpeed,
	color=WindSpeed))+
	
	geom_point(size=3)+
	
	coord_polar()+
	
	scale_color_viridis_c()+
	
	ggtitle("Wind Rose (Polar Plot)")+
	
	theme_minimal()

ax = plt.subplot(111, projection='polar') scatter = ax.scatter(theta, df["WindSpeed"], c=df["WindSpeed"], cmap="plasma", s=50) plt.colorbar(scatter, label="Wind Speed (km/h)") ax.set_title("Wind Rose (Polar Coordinate Plot)") plt.show() print(df.head())	head(wind)

Output





Scenario 4: Hospital Patient Monitoring Dashboard

Python	R

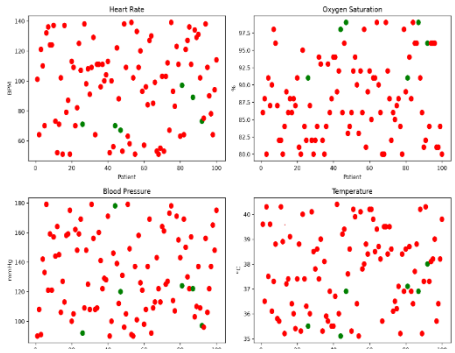
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate sample ICU patient dataset 	
np.random.seed(42)
 	
n = 100
 	
df = pd.DataFrame({
"PatientID": range(1, n + 1),
"HeartRate": np.random.randint(50, 140, n),
"Oxygen": np.random.randint(80, 100, n),
"SystolicBP": np.random.randint(90, 180, n),
"Respiration": np.random.randint(10, 35, n),
	library(ggplot2)
	library(gridExtra)
	
	set.seed(42)
	
	n <- 100
	
	hospital <- data.frame(
	PatientID = 1:n,
	HeartRate = sample(50:140, n, replace = TRUE),
	Oxygen = sample(80:100, n, replace = TRUE),
	SystolicBP = sample(90:180, n, replace = TRUE),
	Respiration = sample(10:35, n, replace = TRUE),

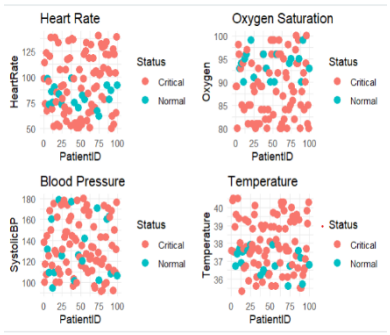
```
    "Temperature":
np.round(np.random.uniform(35.0, 40.5, n), 1)
})

df.to_csv("hospital_data.csv", index=False)

# Function to assign colors based on
normal/abnormal thresholds
def patient_color(hr, oxy, temp):
    if hr < 60 or hr > 100 or oxy < 90 or temp >
38:
        return "red"
    else:
        return "green"

colors = []

for i in range(len(df)):
    colors.append(patient_color(
        df.loc[i, "HeartRate"],
        df.loc[i, "Oxygen"],
        df.loc[i, "Temperature"]
    ))

# Dashboard
plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.scatter(df["PatientID"], df["HeartRate"],
c=colors, s=60)
plt.title("Heart Rate")
plt.xlabel("Patient")
plt.ylabel("BPM")

plt.subplot(2,2,2)
plt.scatter(df["PatientID"], df["Oxygen"],
c=colors, s=60)
plt.title("Oxygen Saturation")
plt.xlabel("Patient")
plt.ylabel("%")

plt.subplot(2,2,3)
plt.scatter(df["PatientID"], df["SystolicBP"],
c=colors, s=60)
plt.title("Blood Pressure")
plt.xlabel("Patient")
plt.ylabel("mmHg")

plt.subplot(2,2,4)
plt.scatter(df["PatientID"], df["Temperature"],
c=colors, s=60)
```

```
  Temperature = round(runif(n, 35.0, 40.5), 1)
)

write.csv(hospital, "hospital_data.csv",
row.names = FALSE)

hospital$Status <- ifelse(
  hospital$HeartRate < 60 |
  hospital$HeartRate > 100 |
  hospital$Oxygen < 90 |
  hospital$Temperature > 38,
  "Critical",
  "Normal"
)

p1 <- ggplot(hospital,
             aes(PatientID, HeartRate, color = Status))
+
  geom_point(size = 3) +
  ggtitle("Heart Rate") +
  theme_minimal()

p2 <- ggplot(hospital,
             aes(PatientID, Oxygen, color = Status)) +
  geom_point(size = 3) +
  ggtitle("Oxygen Saturation") +
  theme_minimal()

p3 <- ggplot(hospital,
             aes(PatientID, SystolicBP, color = Status))
+
  geom_point(size = 3) +
  ggtitle("Blood Pressure") +
  theme_minimal()

p4 <- ggplot(hospital,
             aes(PatientID, Temperature, color =
Status)) +
  geom_point(size = 3) +
  ggtitle("Temperature") +
  theme_minimal()

grid.arrange(p1, p2, p3, p4, ncol = 2)

head(hospital)
```

plt.title("Temperature") plt.xlabel("Patient") plt.ylabel("°C") plt.tight_layout() plt.show() print(df.head())	

Output





Scenario 5: Global Climate Change Study

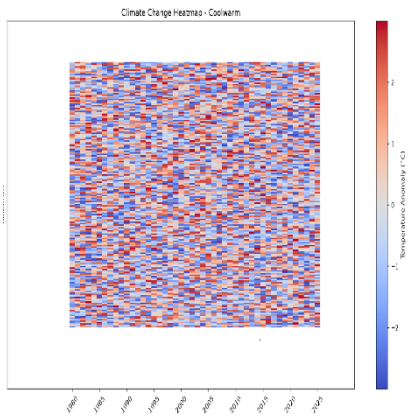
Python	R

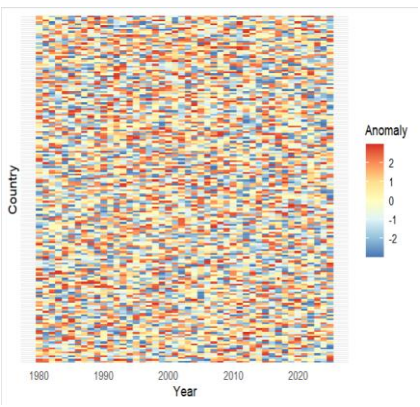
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate Sample Dataset 	
np.random.seed(42)
 	
countries = [f"Country_{i}" for i in range(1, 181)]
years = list(range(1980, 2026))
 	
data = np.random.uniform(-3, 3, (180, len(years)))
 	
df = pd.DataFrame(data, index=countries, columns=years)
 	
df.to_csv("climate_data.csv")
 	
# Heatmap using coolwarm palette 	
plt.figure(figsize=(15,8))
plt.imshow(df, cmap='coolwarm', aspect='auto')
plt.colorbar(label="Temperature Anomaly (°C)")
plt.title("Climate Change Heatmap - Coolwarm")
	library(ggplot2)
	
	# Generate Dataset
	set.seed(42)
	
	countries <- paste("Country",1:180,sep="_")
	years <- 1980:2025
	
	climate <- expand.grid(
	Country=countries,
	Year=years
	)
	
	climate$Anomaly <- runif(nrow(climate),-3,3)
	
	write.csv(climate,"climate_data.csv",row.names=FALSE)
	
	# Heatmap - Gradient2
	ggplot(climate,
	aes(x=Year,
	y=Country,
	fill=Anomaly))+
	
	geom_tile()+
	
	scale_fill_gradient2(
	low="blue",

```
plt.xlabel("Year")
plt.ylabel("Countries")
plt.xticks(range(0, len(years), 5),
years[::5], rotation=45)
plt.yticks([])
plt.show()

# Heatmap using RdBu palette
plt.figure(figsize=(15,8))
plt.imshow(df, cmap='RdBu_r',
aspect='auto')
plt.colorbar(label="Temperature
Anomaly (°C)")
plt.title("Climate Change Heatmap -
RdBu")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.xticks(range(0, len(years), 5),
years[::5], rotation=45)
plt.yticks([])
plt.show()

# Heatmap using PiYG palette
plt.figure(figsize=(15,8))
plt.imshow(df, cmap='PiYG',
aspect='auto')
plt.colorbar(label="Temperature
Anomaly (°C)")
plt.title("Climate Change Heatmap -
PiYG")
plt.xlabel("Year")
plt.ylabel("Countries")
plt.xticks(range(0, len(years), 5),
years[::5], rotation=45)
plt.yticks([])
plt.show()

print(df.head())
```

```
mid="white",
high="red",
midpoint=0)+

ggtitle("Climate Change Heatmap")+

theme_minimal()+

theme(axis.text.y=element_blank())

# Heatmap - Spectral Palette
ggplot(climate,
      aes(Year,
          Country,
          fill=Anomaly))+

geom_tile()+

scale_fill_distiller(palette="Spectral")+

theme_minimal()+

theme(axis.text.y=element_blank())

# Heatmap - RdYlBu Palette
ggplot(climate,
      aes(Year,
          Country,
          fill=Anomaly))+

geom_tile()+

scale_fill_distiller(palette="RdYlBu")+

theme_minimal()+

theme(axis.text.y=element_blank())

head(climate)
```

Output



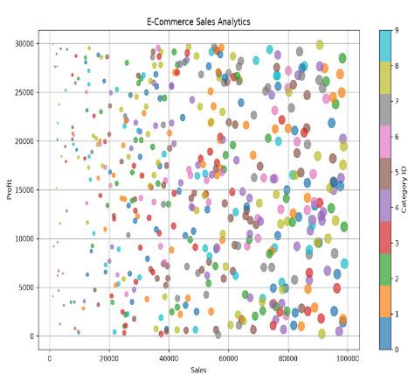


Scenario 6: E-Commerce Sales Analytics

Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate Dataset 	
np.random.seed(42)
 	
n = 500
 	
categories =
["Electronics","Clothing","Furniture","Groceries","Books",
 	
"Toys","Sports","Beauty","Jewelry","Footwear"]
 	
regions =
["North","South","East","West","Central"]
 	
df = pd.DataFrame({
"Category": np.random.choice(categories, n),
"Region": np.random.choice(regions, n),
"Sales": np.random.randint(1000, 100000, n),
"Profit": np.random.randint(100, 30000, n)
})
 	
df.to_csv("ecommerce_sales.csv", index=False)
 	
# Assign numerical values for categories 	
df["CategoryID"] = pd.factorize(df["Category"])[0]
 	
# Scatter Plot 	
plt.figure(figsize=(12,7))
 	
scatter = plt.scatter(
df["Sales"],
df["Profit"],
c=df["CategoryID"],
s=df["Sales"]/700,
cmap="tab10",
alpha=0.7
)
plt.colorbar(scatter, label="Category ID")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.title("E-Commerce Sales Analytics")
plt.grid(True)
plt.show()
 	
print(df.head())
	library(ggplot2)
	
	# Generate Dataset
	set.seed(42)
	
	n <- 500
	
	categories <-
	c("Electronics","Clothing","Furniture","Groceries",
	"Books","Toys","Sports","Beauty",
	"Jewelry","Footwear")
	
	regions <-
	c("North","South","East","West","Central")
	
	sales <- data.frame(
	Category = sample(categories, n, replace = TRUE),
	Region = sample(regions, n, replace = TRUE),
	Sales = sample(1000:100000, n, replace = TRUE),
	Profit = sample(100:30000, n, replace = TRUE)
	)
	
	write.csv(sales, "ecommerce_sales.csv", row.names = FALSE)
	
	print(head(sales))
	
	ggplot(sales,
	aes(x = Sales,
	y = Profit,
	color = Category,
	size = Sales)) +
	
	geom_point(alpha = 0.7) +
	
	ggtitle("E-Commerce Sales Analytics") +
	
	theme_minimal()

Output



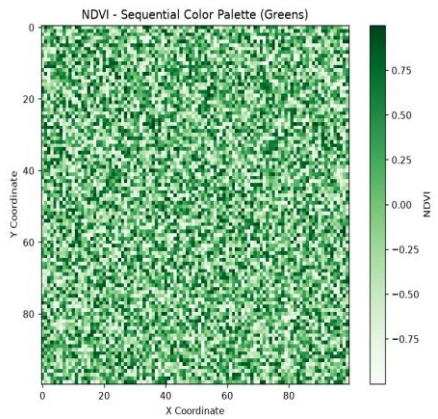
Scenario 7: Satellite Image Vegetation Analysis

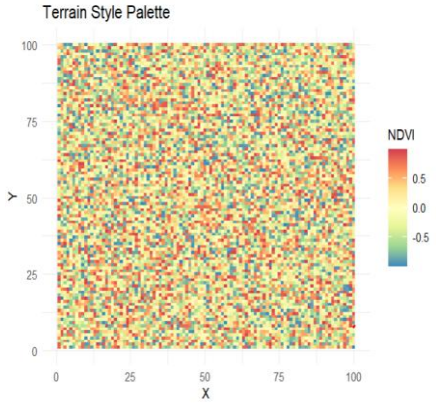
Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate Sample NDVI Dataset 	
np.random.seed(42)
 	
rows = 100
cols = 100
 	
ndvi = np.random.uniform(-1, 1, (rows, cols))
 	
pd.DataFrame(ndvi).to_csv("ndvi_data.csv", index=False)
 	
# Sequential Palette (Greens) 	
plt.figure(figsize=(8,6))
plt.imshow(ndvi, cmap="Greens")
plt.colorbar(label="NDVI")
plt.title("NDVI - Sequential Color Palette (Greens)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
 	
# Diverging Palette (RdYlGn) 	
plt.figure(figsize=(8,6))
plt.imshow(ndvi, cmap="RdYlGn")
plt.colorbar(label="NDVI")
plt.title("NDVI - Diverging Color Palette (RdYlGn)")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
	library(ggplot2)
	
	# Generate Sample Dataset
	set.seed(42)
	
	x <- rep(1:100, each = 100)
	y <- rep(1:100, 100)
	
	ndvi <- data.frame(
	  X = x,
	  Y = y,
	  NDVI = runif(10000, -1, 1)
	)
	
	write.csv(ndvi, "ndvi_data.csv", row.names = FALSE)
	
	print(head(ndvi))
	
	# Sequential Palette
	ggplot(ndvi,
	  aes(X, Y, fill = NDVI)) +
	  geom_tile() +
	  scale_fill_gradient(
	  low = "white",
	  high = "darkgreen"
	  ) +
	  ggtitle("Sequential Palette (Greens)") +
	  theme_minimal()
	
	# Diverging Palette
	ggplot(ndvi,
	  aes(X, Y, fill = NDVI)) +
	  geom_tile() +
	  scale_fill_gradient2(

# Terrain Palette 	
plt.figure(figsize=(8,6))
plt.imshow(ndvi, cmap="terrain")
plt.colorbar(label="NDVI")
plt.title("NDVI - Terrain Color Palette")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()
 	
print("First 5 Rows of NDVI Data")
print(pd.DataFrame(ndvi).head())
	low = "red",
	mid = "yellow",
	high = "darkgreen",
	midpoint = 0
	) +
	ggtitle("Diverging Palette (RdYlGn Style)") +
	theme_minimal()
	
	# Terrain Palette
	ggplot(ndvi,
	aes(X, Y, fill = NDVI)) +
	geom_tile() +
	scale_fill_distiller(palette = "Spectral") +
	ggtitle("Terrain Style Palette") +
	theme_minimal()

Output





Scenario 8: Cryptocurrency Market Volatility

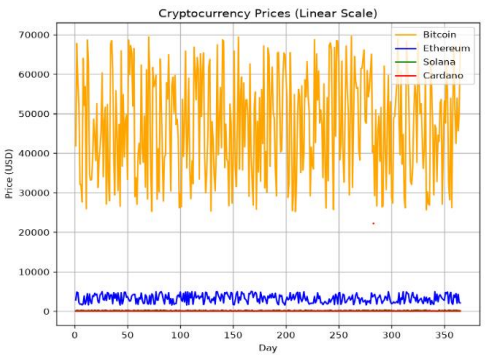
Python	R

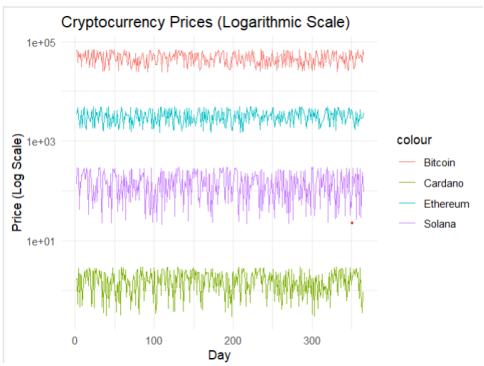
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Generate Sample Dataset 	
np.random.seed(42)
days = np.arange(1, 366)
bitcoin = np.random.uniform(25000, 70000, 365)
ethereum = np.random.uniform(1500, 5000, 365)
solana = np.random.uniform(20, 300, 365)
cardano = np.random.uniform(0.3, 3.0, 365)
crypto = pd.DataFrame({
"Day": days,
"Bitcoin": bitcoin,
"Ethereum": ethereum,
"Solana": solana,
"Cardano": cardano
	library(ggplot2)
	
	# Generate Dataset
	set.seed(42)
	
	days <- 1:365
	
	crypto <- data.frame(
	Day = days,
	Bitcoin = runif(365,25000,70000),
	Ethereum = runif(365,1500,5000),
	Solana = runif(365,20,300),
	Cardano = runif(365,0.3,3)
	)
	
	write.csv(crypto,"crypto_data.csv",row.names=FALSE)
	
	print(head(crypto))

```
})
crypto.to_csv("crypto_data.csv",
index=False)
print(crypto.head())
plt.figure(figsize=(12,6))
plt.plot(crypto["Day"], crypto["Bitcoin"],
color="orange", label="Bitcoin")
plt.plot(crypto["Day"],
crypto["Ethereum"], color="blue",
label="Ethereum")
plt.plot(crypto["Day"], crypto["Solana"],
color="green", label="Solana")
plt.plot(crypto["Day"], crypto["Cardano"],
color="red", label="Cardano")
plt.title("Cryptocurrency Prices (Linear
Scale)")
plt.xlabel("Day")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(12,6))
plt.plot(crypto["Day"], crypto["Bitcoin"],
color="orange", label="Bitcoin")
plt.plot(crypto["Day"],
crypto["Ethereum"], color="blue",
label="Ethereum")
plt.plot(crypto["Day"], crypto["Solana"],
color="green", label="Solana")
plt.plot(crypto["Day"], crypto["Cardano"],
color="red", label="Cardano")
plt.yscale("log")
plt.title("Cryptocurrency Prices
(Logarithmic Scale)")
plt.xlabel("Day")
plt.ylabel("Price (Log Scale)")
plt.legend()
plt.grid(True)
plt.show()
```

```
# ---------- Linear Scale ----------

ggplot(crypto) +
  geom_line(aes(Day, Bitcoin, color="Bitcoin")) +
  geom_line(aes(Day, Ethereum, color="Ethereum")) +
  geom_line(aes(Day, Solana, color="Solana")) +
  geom_line(aes(Day, Cardano, color="Cardano")) +
  labs(title="Cryptocurrency Prices (Linear Scale)",
       x="Day",
       y="Price (USD)") +
  theme_minimal()

# ---------- Logarithmic Scale ----------

ggplot(crypto) +
  geom_line(aes(Day, Bitcoin, color="Bitcoin")) +
  geom_line(aes(Day, Ethereum, color="Ethereum")) +
  geom_line(aes(Day, Solana, color="Solana")) +
  geom_line(aes(Day, Cardano, color="Cardano")) +
  scale_y_log10() +
  labs(title="Cryptocurrency Prices (Logarithmic
Scale)",
       x="Day",
       y="Price (Log Scale)") +
  theme_minimal()
```

Output





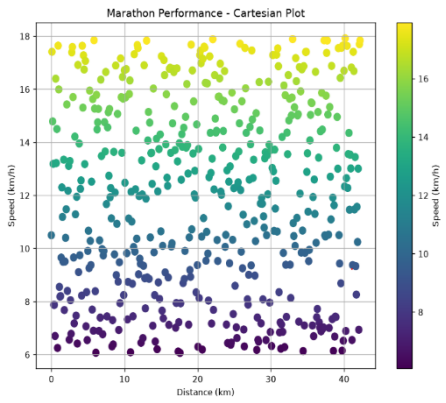
Scenario 9: Marathon Runner Performance Analysis

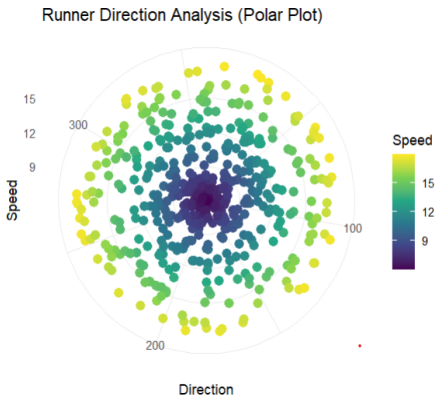
Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
 	
# Generate Sample Dataset 	
np.random.seed(42)
 	
n = 500
 	
runner = pd.DataFrame({
"Distance": np.linspace(0, 42.195, n),
"Speed": np.random.uniform(6, 18, n),
"HeartRate": np.random.randint(90, 190, n),
"Elevation": np.random.randint(0, 500, n),
"Direction": np.random.uniform(0, 360, n)
})
 	
runner.to_csv("marathon_data.csv", index=False)
 	
print(runner.head())
 	
# ---------- Cartesian Plot ---------- 	
plt.figure(figsize=(10,6))
 	
scatter = plt.scatter(
runner["Distance"],
runner["Speed"],
c=runner["Speed"],
cmap="viridis",
s=60
)
 	
plt.colorbar(scatter, label="Speed (km/h)")
plt.title("Marathon Performance - Cartesian Plot")
plt.xlabel("Distance (km)")
plt.ylabel("Speed (km/h)")
plt.grid(True)
 	
plt.show()
	library(ggplot2)
	
	# Generate Dataset
	set.seed(42)
	
	n <- 500
	
	runner <- data.frame(
	Distance = seq(0,42.195,length.out=n),
	Speed = runif(n,6,18),
	HeartRate = sample(90:190,n,replace=TRUE),
	Elevation = sample(0:500,n,replace=TRUE),
	Direction = runif(n,0,360)
	)
	
	write.csv(runner,"marathon_data.csv",row.names=FALSE)
	
	print(head(runner))
	
	# ---------- Cartesian Plot ----------
	
	ggplot(runner,
	aes(x=Distance,
	y=Speed,
	color=Speed))+
	
	geom_point(size=3)+
	
	scale_color_viridis_c()+
	
	labs(
	title="Marathon Performance (Cartesian Plot)",
	x="Distance (km)",
	y="Speed (km/h)"
	)+
	
	theme_minimal()
	
	# ---------- Polar Plot ----------
	
	ggplot(runner,
	aes(x=Direction,
	y=Speed,
	color=Speed))+
	
	geom_point(size=3)+
	
	coord_polar()+

# ---------- Polar Plot ---------- 	
 	
theta =
np.deg2rad(runner["Direction"])
 	
fig = plt.figure(figsize=(8,8))
ax = plt.subplot(111,
projection="polar")
 	
scatter = ax.scatter(
  theta,
  runner["Speed"],
  c=runner["Speed"],
  cmap="plasma",
  s=40
)
 	
plt.colorbar(scatter, label="Speed
(km/h)")
ax.set_title("Runner Direction Analysis
(Polar Plot)")
 	
plt.show()
	scale_color_viridis_c()+
	
	labs(
	title="Runner Direction Analysis (Polar Plot)"
	)+
	
	theme_minimal()

Output





Scenario 10: COVID-19 Vaccination Campaign Dashboard

Python	R

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Generate Sample Dataset 	
np.random.seed(42)
districts = [f"District_{i}" for i in range(1, 31)]
age_groups = ["0-17", "18-44", "45-60", "60+"]
vaccines = ["Covaxin", "Covishield", "Sputnik"]
	library(ggplot2)
	# Generate Dataset
	set.seed(42)
	districts <- paste("District", 1:30, sep = "_")
	agegroups <- c("0-17","18-44","45-60","60+")
	vaccines <- c("Covaxin","Covishield","Sputnik")
	
	vaccination <- expand.grid(
	  District = districts,

```
rows = []
for district in districts:
    for age in age_groups:
        for vaccine in vaccines:
            rows.append([
                district,
                age,
                vaccine,
                np.random.randint(500, 10000)
            ])
df = pd.DataFrame(rows, columns=[
    "District",
    "AgeGroup",
    "Vaccine",
    "Vaccinated"
])
df.to_csv("vaccination_data.csv", index=False)
print(df.head())
# ---------- Version 1 ----------
# Blue Color Palette
plt.figure(figsize=(12,6))
plt.bar(df["District"][:30],
        df["Vaccinated"][:30],
        color="steelblue")
plt.xticks(rotation=90)
plt.title("COVID-19 Vaccination Dashboard (Blue
Palette)")
plt.xlabel("District")
plt.ylabel("Vaccinated Population")
plt.tight_layout()
plt.show()
# ---------- Version 2 ----------
# Green Palette with Highlight
colors = []
for value in df["Vaccinated"][:30]:
    if value > 7000:
        colors.append("red")
    else:
        colors.append("green")
plt.figure(figsize=(12,6))
plt.bar(df["District"][:30],
        df["Vaccinated"][:30],
        color=colors)
plt.xticks(rotation=90)
plt.title("COVID-19 Vaccination Dashboard
(Highlighting)")
plt.xlabel("District")
plt.ylabel("Vaccinated Population")
plt.tight_layout()
plt.show()
# ---------- Heatmap ----------
```

```
  AgeGroup = agegroups,
  Vaccine = vaccines
)
vaccination$Vaccinated <- sample(
  500:10000,
  nrow(vaccination),
  replace = TRUE
)
write.csv(vaccination,
          "vaccination_data.csv",
          row.names = FALSE)
print(head(vaccination))
# ---------- Version 1 ----------
ggplot(vaccination,
       aes(x = District,
           y = Vaccinated,
           fill = Vaccinated)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue"
  ) +
  labs(
    title = "COVID-19 Vaccination Dashboard",
    x = "District",
    y = "Vaccinated Population"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(
    angle = 90,
    hjust = 1
  ))
vaccination$Status <- ifelse(
  vaccination$Vaccinated > 7000,
  "High",
  "Normal"
)
ggplot(vaccination,
       aes(x = District,
           y = Vaccinated,
           fill = Status)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = c(
    "High" = "red",
    "Normal" = "green"
  )) +
  labs(
    title = "Vaccination Dashboard with
Highlighting",
    x = "District",
    y = "Vaccinated Population"
```

```
pivot = df.pivot_table(
   values="Vaccinated",
   index="District",
   columns="AgeGroup",
   aggfunc="sum"
)
plt.figure(figsize=(8,10))
plt.imshow(pivot,
         cmap="YlGnBu",
         aspect="auto")
plt.colorbar(label="Vaccinated")
plt.title("Vaccination Heatmap")
plt.xticks(range(len(pivot.columns)),
         pivot.columns)
plt.yticks(range(len(pivot.index)),
         pivot.index)
plt.show()
```

```
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(
    angle = 90,
    hjust = 1
  ))
ggplot(vaccination,
       aes(x = AgeGroup,
           y = District,
           fill = Vaccinated)) +
  geom_tile() +
  scale_fill_gradient(
    low = "yellow",
    high = "darkgreen"
  ) +
  labs(
    title = "Vaccination Heatmap"
  ) +
  theme_minimal()
```

Output